Speech and Language Processing, selected pages for class 4

Daniel Jurafsky and James H. Martin, Speech and Language Processing, 3rd edition draft, August 2026. The whole draft is free at https://web.stanford.edu/~jurafsky/slp3/ and the authors allow its use in classes. These pages are for CSE 4811 / DS 4213 Natural Language Processing, Fall 2026, Department of Computer Science and Engineering, United International University. Page numbers are the book's.

- Section 6.5, Embeddings as the input to neural net classifiers, including the feedforward neural language model, pages 163 to 168
- Section 5.5, Word2vec, pages 136 to 142
- Section 5.7, Semantic properties of embeddings, including analogies, pages 143 to 145

6.4.2 Vectorizing for parallelizing inference

While Eq. 6.19 shows how to classify a single example x, in practice we want to efficiently classify an entire test set of m examples. We do this by vectorizing the process, just as we saw with logistic regression; instead of using for-loops to go through each example, we'll use matrix multiplication to do the entire computation of an entire test set at once. First, we pack all the input feature vectors for each input x into a single input matrix $\mathbf{X}$, with each row i a row vector consisting of the features for input example $x^{(i)}$ (i.e., the vector $\mathbf{x}^{(i)}$). If the dimensionality of our input feature vector is d, $\mathbf{X}$ will be a matrix of shape $[m \times d]$.

Because we are now modeling each input as a row vector rather than a column vector, we also need to slightly modify Eq. 6.19. $\mathbf{X}$ is of shape $[m \times d]$ and $\mathbf{W}$ is of shape $[d_h \times d]$, so we'll reorder how we multiply $\mathbf{X}$ and $\mathbf{W}$ and transpose $\mathbf{W}$ so they correctly multiply to yield a matrix $\mathbf{H}$ of shape $[m \times d_h]$. [1]

The bias vector $\mathbf{b}$ from Eq. 6.19 of shape $[1 \times d_h]$ will now have to be replicated into a matrix of shape $[m \times d_h]$. We'll need to similarly reorder the next step and transpose $\mathbf{U}$. Finally, our output matrix $\hat{\mathbf{Y}}$ will be of shape $[m \times 3]$ (or more generally $[m \times d_o]$, where d_o is the number of output classes), with each row i of our output matrix $\hat{\mathbf{Y}}$ consisting of the output vector $\hat{\mathbf{y}}^{(i)}$. Here are the final equations for computing the output class distribution for an entire test set:

$$\begin{aligned} \mathbf{H} &= \sigma(\mathbf{X}\mathbf{W}^\intercal + \mathbf{b}) \\ \mathbf{Z} &= \mathbf{H}\mathbf{U}^\intercal \\ \hat{\mathbf{Y}} &= \text{softmax}(\mathbf{Z}) \end{aligned} \tag{6.20}$$

In this book, we'll sometimes see orderings like $\mathbf{W}\mathbf{X} + \mathbf{b}$ and sometimes $\mathbf{X}\mathbf{W} + \mathbf{b}$. That's why it's always important to be very aware of the shapes of your weight matrices participating in any given equation.

6.5 Embeddings as the input to neural net classifiers

While hand-built features are a traditional way to design classifiers, most applications of neural networks for NLP don't use hand-built human-engineered features as inputs. Instead, we draw on deep learning's ability to learn features from the data by representing tokens as embeddings. For this section we'll represent each token by its static word2vec or GloVe embeddings that we saw how to compute in Chapter 5. By static embedding, we mean that each token is represented by a fixed vector that we train once, and then just put into a big dictionary. When we want to refer to that token, we grab its embedding out of the dictionary.

However when we apply neural models to the task of language modeling (as we'll see in Chapter 7) the situation is more complex, and we'll use a more powerful kind of embedding called a *contextual embedding*. Contextual embeddings are different for each time a word occurs in a different context. Furthermore, we'll have the network learn these embeddings as part of the task of word prediction.

So let's explore the text classification domain above, but using static embeddings as features instead of the hand-designed features. Let's focus on the inference stage,

[1] Note that we could have kept the original order of our products if we had instead made our input matrix $\mathbf{X}$ represent each input as a column vector instead of a row vector, making it of shape $[d \times m]$. But representing inputs as row vectors is convenient and common in neural network models.

in which we have already learned embeddings for all the input tokens. An embedding is a vector of dimension d that represents the input token. The dictionary of static embeddings in which we store these embeddings is the **embedding matrix** $\mathbf{E}$. Each row of the embedding matrix represents each token of the vocabulary V as a (row) vector of dimensionality d. Since $\mathbf{E}$ has a row for each of the $|V|$ tokens in the vocabulary, $\mathbf{E}$ has shape $[|V| \times d]$. This embedding matrix $\mathbf{E}$ plays a role whenever we are using embeddings as input to neural NLP systems, including in the transformer-based large language models we will introduce over the next chapters.

embedding matrix

Given an input token string like `dessert was great` we first convert the tokens into vocabulary indices (these were created when we first tokenized the input using BPE or SentencePiece). So the representation of `dessert was great` might be $\mathbf{w} = [3, 524, 902]$. Next we use indexing to select the corresponding rows from $\mathbf{E}$ (row 3, row 524, row 902).

Another way to think about selecting token embeddings from the embedding matrix is to represent input tokens as one-hot vectors of shape $[1 \times |V|]$, i.e., with one dimension for each word in the vocabulary. Recall that in a **one-hot vector** all the elements are 0 except one, the element whose dimension is the word's index in the vocabulary, which has value 1. So if the word "dessert" has index 3 in the vocabulary, $x_3 = 1$, and $x_i = 0 \ \forall i \neq 3$, as shown here:

one-hot vector

```
[0 0 1 0 0 0 0 ... 0 0 0 0]
 1 2 3 4 5 6 7 ...  ... |V|
```

Multiplying by a one-hot vector that has only one non-zero element $x_i = 1$ simply selects out the relevant row vector for word i, resulting in the embedding for word i, as depicted in Fig. 6.11.

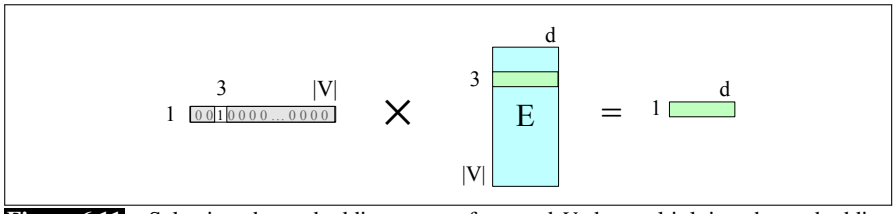


Figure 6.11 Selecting the embedding vector for word V_3 by multiplying the embedding matrix $\mathbf{E}$ with a one-hot vector with a 1 in index 3.

We can extend this idea to represent the entire input token sequence as a matrix of one-hot vectors, one for each of the N input positions as shown in Fig. 6.12.

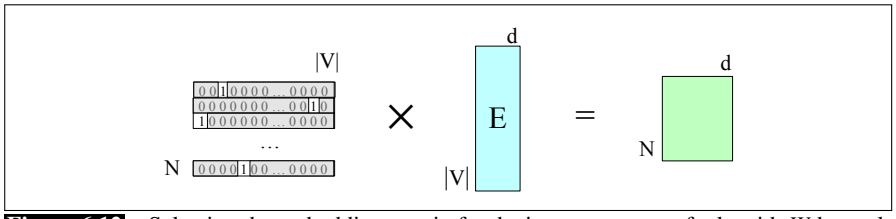


Figure 6.12 Selecting the embedding matrix for the input sequence of token ids W by multiplying a one-hot matrix corresponding to W by the embedding matrix $\mathbf{E}$.

We now need to classify this input of N $[1 \times d]$ embeddings, representing a window of N tokens, into a single class (like positive or negative).

There are two common ways to pass embeddings to a classifier: **concatenation** and **pooling**. First, we can take this input of shape $[N \times d]$ and reshape it by **concatenating** all the input vectors into one very long vector of shape $[1 \times dN]$. Then

we pass this input to our classifier and let it make its decision. This gives us lots of information, at the cost of using a pretty large network. Second, we can **pool** the N embeddings into a single embedding and then pass that single pooled embedding to the classifier. Pooling gives us less information than would have been present in all the original embeddings, but has the advantage of being small and efficient and is especially useful in tasks for which we don't care as much about the original word order. Let's give an example of each: pooling for the sentiment task, and concatenation for the language modeling task.

pool

Pooling input embeddings for sentiment So let's begin with seeing how pooling can work for the sentiment classification task. The intuition of pooling is that for sentiment, the exact position of the input (is some word like `great` the first word? the second word?) is less important than the identity of the word itself.

A pooling function is a way to turn a set of embeddings into a single embedding.

For example, for a text with N input words/tokens $w_1, ..., w_N$, we want to turn the N row embeddings $\mathbf{e}(w_1), ..., \mathbf{e}(w_N)$ (each of dimensionality d) into a single embedding also of dimensionality d.

mean-pooling

There are various ways to pool. The simplest is **mean-pooling**: taking the mean by summing the embeddings and then dividing by N:

$$\mathbf{x}_{\text{mean}} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{e}(w_i) \tag{6.21}$$

Here are the equations for this classifier assuming mean pooling:

$$\begin{aligned}\mathbf{x} &= \text{mean}(\mathbf{e}(w_1), \mathbf{e}(w_2), \ldots, \mathbf{e}(w_n))\\ \mathbf{h} &= \sigma(\mathbf{x}\mathbf{W} + \mathbf{b})\\ \mathbf{z} &= \mathbf{h}\mathbf{U}\\ \hat{\mathbf{y}} &= \text{softmax}(\mathbf{z})\end{aligned} \tag{6.22}$$

The architecture is sketched in Fig. 6.13, where we also give the shapes for all the relevant matrices (note that we're using row vectors now).

max-pooling

There are many other options for pooling, like **max-pooling**, in which case for each dimension we take the element-wise max over all the inputs. The element-wise max of a set of N vectors is a new vector whose kth element is the max of the kth elements of all the N vectors.

Concatenating input embeddings for language modeling For sentiment analysis we saw how to generate an output vector with probabilities over three classes: positive, negative, or neutral, given as input a window of N input tokens, by first pooling those token embeddings into a single embedding vector.

Now let's consider **language modeling**: predicting upcoming words from prior words. In this task we are given the same window of N input tokens, but our task now is to predict the next token that should follow the window. We'll sketch a simple feedforward neural language model, drawing on an algorithm first introduced by Bengio et al. (2003). The feedforward language model introduces many of the important concepts of large language modeling that we will return to in Chapter 7.

Neural language models have many advantages over the n-gram language models of Chapter 3. Neural language models can handle much longer histories, can generalize better over contexts of similar words, and are far more accurate at word-prediction. On the other hand, neural net language models are slower, more complex, need vast amounts of energy to train, and are less interpretable than n-gram models, so for some smaller tasks an n-gram language model is still the right tool.

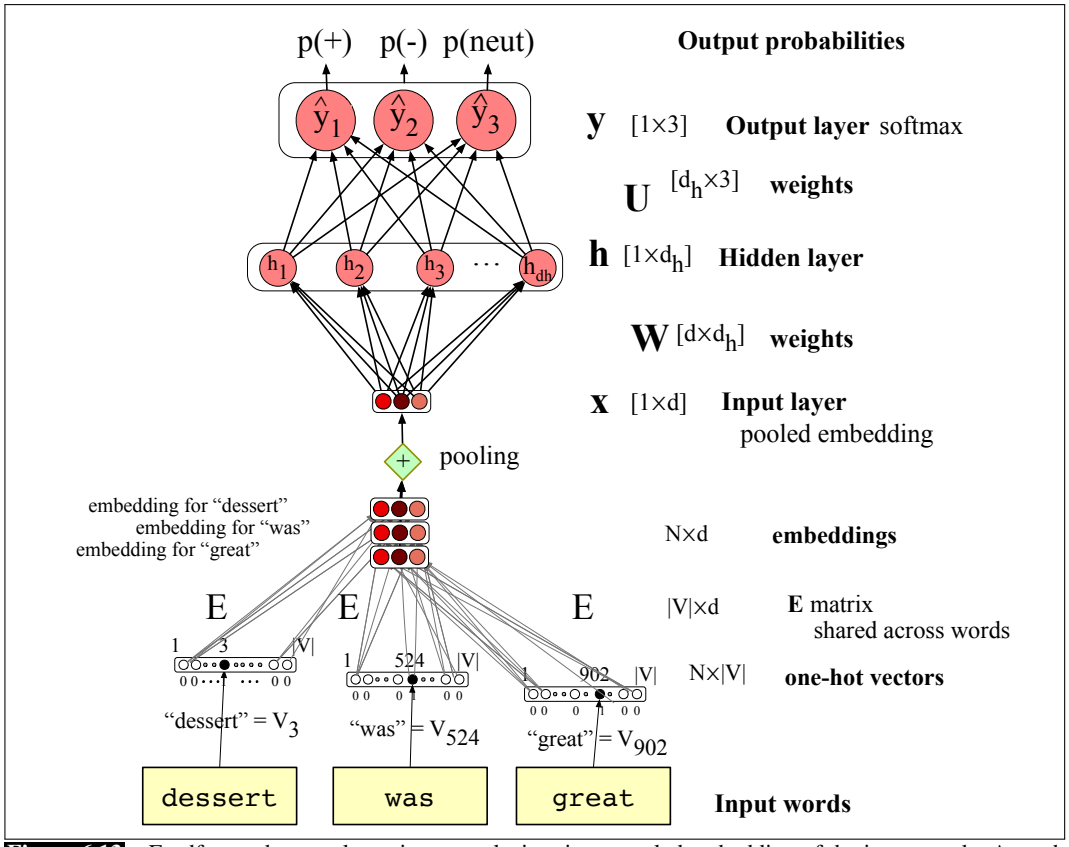


Figure 6.13 Feedforward network sentiment analysis using a pooled embedding of the input words. At each timestep the network computes a d-dimensional embedding for each context word (by multiplying a one-hot vector by the embedding matrix $\mathbf{E}$), and pools the resulting N embeddings to get a single embedding that represents the context window as the layer $\mathbf{x}$.

A feedforward neural language model is a feedforward network that takes as input at time t a representation of some number of previous words (w_{t-1}, w_{t-2}, etc.) and outputs a probability distribution over possible next words. Thus—like the n-gram LM—the feedforward neural LM approximates the probability of a word given the entire prior context $P(w_t|w_{1:t-1})$ by approximating based on the $N-1$ previous words:

$$P(w_t|w_1,\ldots,w_{t-1}) \approx P(w_t|w_{t-N+1},\ldots,w_{t-1}) \tag{6.23}$$

In the following examples we'll use a 4-gram example, so we'll show a neural net to estimate the probability $P(w_t = i|w_{t-3}, w_{t-2}, w_{t-1})$.

Neural language models represent words in this prior context by their **embeddings**, rather than just by their word identity as used in n-gram language models. Using embeddings allows neural language models to generalize better to unseen data. For example, suppose we've seen this sentence in training:

I have to make sure that the cat gets fed.

but have never seen the words "gets fed" after the word "dog". Our test set has the prefix "I forgot to make sure that the dog gets". What's the next word? An n-gram language model will predict "fed" after "that the cat gets", but not after "that the dog gets". But a neural LM, knowing that "cat" and "dog" have similar embeddings, will

be able to generalize from the "cat" context to assign a high enough probability to "fed" even after seeing "dog".

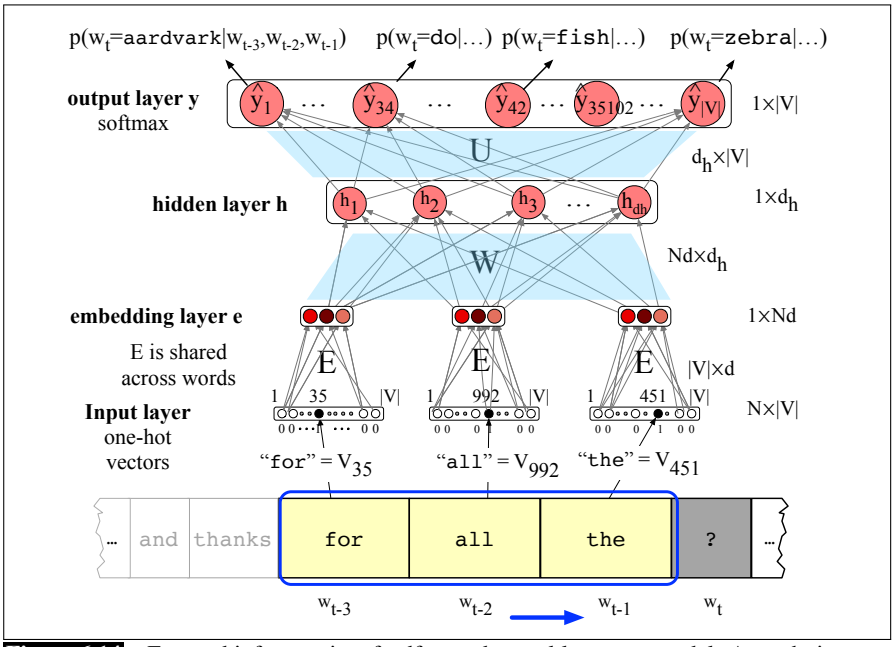


Figure 6.14 Forward inference in a feedforward neural language model. At each timestep t the network computes a d-dimensional embedding for each of the $N = 3$ context tokens (by multiplying a one-hot vector by the embedding matrix **E**), and concatenates the three to get the embedding **e**. This embedding **e** is multiplied by weight matrix **W** and then an activation function is applied element-wise to produce the hidden layer **h**, which is then multiplied by another weight matrix **U**. A softmax layer predicts at each output node i the probability that the next word w_t will be vocabulary word V_i. We show the context window size N as 3 just to fit on the page, but in practice language modeling requires a much longer context.

This prediction task requires an output vector that expresses $|V|$ probabilities: one probability value for each possible next token. We might have a vocabulary between 60,000 and 300,000 tokens, so the output vector for the task of language modeling is much longer than 3. Another difference for language modeling is that instead of pooling the embeddings of the N input tokens to create a single embedding, we concatenate the inputs into one very long input vector. To predict the next token, it helps to know each of the preceding tokens and what order they were in.

Fig. 6.14 shows the language modeling task, sketched with a very short context window of $N = 3$ just to fit on the page. These 3 embedding vectors are concatenated to produce **e**, the embedding layer. This is multiplied by a weight matrix **W** to produce a hidden layer, and another weight matrix **U** to produce an output layer whose softmax gives a probability distribution over words. For example y_{42}, the value of output node 42, is the probability of the next word w_t being V_{42}, the vocabulary word with index 42 (which is the word 'fish' in our example).

The equations for a simple feedforward neural language model with a window

size of 3, given one-hot input vectors for each input context word, are:

$$\begin{aligned}\mathbf{e} &= [\mathbf{E}\mathbf{x}_{t-3}; \mathbf{E}\mathbf{x}_{t-2}; \mathbf{E}\mathbf{x}_{t-1}] \\ \mathbf{h} &= \sigma(\mathbf{W}\mathbf{e} + \mathbf{b}) \\ \mathbf{z} &= \mathbf{U}\mathbf{h} \\ \hat{\mathbf{y}} &= \text{softmax}(\mathbf{z}) \end{aligned} \tag{6.24}$$

Note that we use semicolons to mean concatenation of vectors, so we form the embedding layer $\mathbf{e}$ by concatenating the 3 embeddings for the three context vectors.

We'll return to this idea of using neural networks to do language modeling in Chapter 7 when we introduce transformer language models.

6.6 Training Neural Nets

A feedforward neural net is an instance of supervised machine learning in which we know the correct output y for each observation x. What the system produces, via Eq. 6.13, is $\hat{y}$, the system's estimate of the true y. The goal of the training procedure is to learn parameters $\mathbf{W}^{[i]}$ and $\mathbf{b}^{[i]}$ for each layer i that make $\hat{y}$ for each training observation as close as possible to the true y.

In general, we do all this by drawing on the methods we introduced in Chapter 4 for logistic regression, so the reader should be comfortable with that chapter before proceeding. We'll explore the algorithm on simple generic networks rather than networks designed for sentiment or language modeling.

First, we'll need a **loss function** that models the distance between the system output and the gold output, and it's common to use the loss function used for logistic regression, the **cross-entropy loss**.

Second, to find the parameters that minimize this loss function, we'll use the **gradient descent** optimization algorithm introduced in Chapter 4.

Third, gradient descent requires knowing the **gradient** of the loss function, the vector that contains the partial derivative of the loss function with respect to each of the parameters. In logistic regression, for each observation we could directly compute the derivative of the loss function with respect to an individual w or b. But for neural networks, with millions of parameters in many layers, it's much harder to see how to compute the partial derivative of some weight in layer 1 when the loss is attached to some much later layer. How do we partial out the loss over all those intermediate layers? The answer is the algorithm called **error backpropagation** or **backward differentiation**.

6.6.1 Loss function

cross-entropy loss

The **cross-entropy loss** that is used in neural networks is the same one we saw for logistic regression. If the neural network is being used as a binary classifier, with the sigmoid at the final layer, the loss function is the same logistic regression loss we saw in Eq. 4.19:

$$L_{CE}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1-y)\log(1-\hat{y})] \tag{6.25}$$

If we are using the network to classify into 3 or more classes, the loss function is exactly the same as the loss for multinomial regression that we saw in Chapter 4 on

$$\cos(\text{cherry},\text{information}) = \frac{442*5+8*3982+2*3325}{\sqrt{442^2+8^2+2^2}\sqrt{5^2+3982^2+3325^2}} = .018$$

$$\cos(\text{digital},\text{information}) = \frac{5*5+1683*3982+1670*3325}{\sqrt{5^2+1683^2+1670^2}\sqrt{5^2+3982^2+3325^2}} = .996$$

The model decides that *information* is way closer to *digital* than it is to *cherry*, a result that seems sensible. Fig. 5.5 shows a visualization.

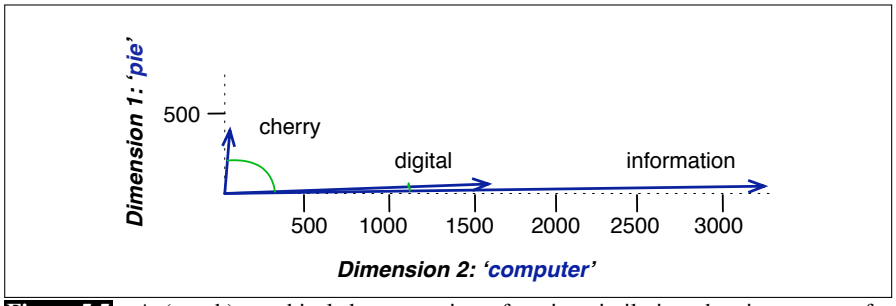


Figure 5.5 A (rough) graphical demonstration of cosine similarity, showing vectors for three words (*cherry*, *digital*, and *information*) in the two dimensional space defined by counts of the words *computer* and *pie* nearby. The figure doesn't show the cosine, but it highlights the angles; note that the angle between *digital* and *information* is smaller than the angle between *cherry* and *information*. When two vectors are more similar, the cosine is larger but the angle is smaller; the cosine has its maximum (1) when the angle between two vectors is smallest (0°); the cosine of all other angles is less than 1.

Cosine similarity can be used to estimate word similarity, for tasks like finding word paraphrases, tracking changes in word meaning, or automatically discovering meanings of words in different corpora. For example, we can find the 10 most similar words to any target word w by computing the cosines between w and each of the $|V|-1$ other words, sorting, and looking at the top 10.

5.5 Word2vec

In the previous sections we saw how to represent a word as a sparse, long vector with dimensions corresponding to words in the vocabulary. We now introduce a more powerful word representation: **embeddings**, short dense vectors. Unlike the vectors we've seen so far, embeddings are **short**, with number of dimensions d ranging from 50-1000, rather than the much larger vocabulary size $|V|$. These d dimensions don't have a clear interpretation. And the vectors are **dense**: instead of vector entries being sparse, mostly-zero counts or functions of counts, the values will be real-valued numbers that can be negative.

It turns out that dense vectors work better in every NLP task than sparse vectors. While we don't completely understand all the reasons for this, we have some intuitions. Representing words as 300-dimensional dense vectors requires our classifiers to learn far fewer weights than if we represented words as 50,000-dimensional vectors, and the smaller parameter space possibly helps with generalization and avoiding overfitting. Dense vectors may also do a better job of capturing synonymy. For example, in a sparse vector representation, dimensions for synonyms like *car* and

automobile are distinct and unrelated; sparse vectors may thus fail to capture the similarity between a word with *car* as a neighbor and a word with *automobile* as a neighbor.

skip-gram
SGNS
word2vec

In this section we introduce one method for computing embeddings: **skip-gram with negative sampling**, sometimes called **SGNS**. The skip-gram algorithm is one of two algorithms in a software package called **word2vec**, and so sometimes the algorithm is loosely referred to as word2vec (Mikolov et al. 2013a, Mikolov et al. 2013b). The word2vec methods are fast, efficient to train, and easily available online with code and pretrained embeddings. Word2vec embeddings are **static embeddings**, meaning that the method learns one fixed embedding for each word in the vocabulary. In Chapter 9 we'll introduce methods for learning dynamic **contextual embeddings** like the popular family of **BERT** representations, in which the vector for each word is different in different contexts.

static embeddings

The intuition of word2vec is that instead of counting how often each context word c occurs near, say, *apricot*, we'll instead train a classifier on a binary prediction task: "Is word c likely to show up near *apricot*?" We don't actually care about this prediction task; instead we'll take the learned classifier *weights* as the word embeddings.

The revolutionary intuition here is that we can just use running text as implicitly supervised training data for such a classifier; a word c that occurs near the target word *apricot* acts as gold 'correct answer' to the question "Is word c likely to show up near *apricot*?" This method, often called **self-supervision**, avoids the need for any sort of hand-labeled supervision signal. This idea was first proposed in the task of neural language modeling, when Bengio et al. (2003) and Collobert et al. (2011) showed that a neural language model (a neural network that learned to predict the next word from prior words) could just use the next word in running text as its supervision signal, and could be used to learn an embedding representation for each word as part of doing this prediction task.

self-supervision

We'll see how to do neural networks in the next chapter, but word2vec is a much simpler model than the neural network language model, in two ways. First, word2vec simplifies the task (making it binary classification instead of word prediction). Second, word2vec simplifies the architecture (training a logistic regression classifier instead of a multi-layer neural network with hidden layers that demand more sophisticated training algorithms). The intuition of skip-gram is:

1. Treat the target word and a neighboring context word as positive examples.
2. Randomly sample other words in the lexicon to get negative samples.
3. Use logistic regression to train a classifier to distinguish those two cases.
4. Use the learned weights as the embeddings.

5.5.1 The classifier

Let's start by thinking about the classification task, and then turn to how to train. Imagine a sentence like the following, with a target word *apricot*, and assume we're using a window of ± 2 context words:

```
... lemon,  a [tablespoon of apricot jam,      a] pinch ...
               c1         c2    w     c3       c4
```

Our goal is to train a classifier such that, given a tuple (w, c) of a target word w paired with a candidate context word c (for example (*apricot*, *jam*), or perhaps

(*apricot*, *aardvark*)) it will return the probability that c is a real context word (true for *jam*, false for *aardvark*):

$$P(+|w,c) \tag{5.11}$$

The probability that word c is not a real context word for w is just 1 minus Eq. 5.11:

$$P(-|w,c) = 1 - P(+|w,c) \tag{5.12}$$

How does the classifier compute the probability P? The intuition of the skip-gram model is to base this probability on embedding similarity: a word is likely to occur near the target if its embedding vector is similar to the target embedding. To compute similarity between these dense embeddings, we rely on the intuition that two vectors are similar if they have a high **dot product** (after all, cosine is just a normalized dot product). In other words:

$$\text{Similarity}(w,c) \approx \mathbf{c} \cdot \mathbf{w} \tag{5.13}$$

The dot product $\mathbf{c} \cdot \mathbf{w}$ is not a probability, it's just a number ranging from $-\infty$ to ∞ (since the elements in word2vec embeddings can be negative, the dot product can be negative). To turn the dot product into a probability, we'll use the **logistic** or **sigmoid** function $\sigma(x)$, the fundamental core of logistic regression:

$$\sigma(x) = \frac{1}{1+\exp(-x)} \tag{5.14}$$

We model the probability that word c is a real context word for target word w as:

$$P(+|w,c) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1+\exp(-\mathbf{c} \cdot \mathbf{w})} \tag{5.15}$$

The sigmoid function returns a number between 0 and 1, but to make it a probability we'll also need the total probability of the two possible events (c is a context word, and c isn't a context word) to sum to 1. We thus estimate the probability that word c is not a real context word for w as:

$$\begin{aligned} P(-|w,c) &= 1 - P(+|w,c) \\ &= \sigma(-\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1+\exp(\mathbf{c} \cdot \mathbf{w})} \end{aligned} \tag{5.16}$$

Equation 5.15 gives us the probability for one word, but there are many context words in the window. Skip-gram makes the simplifying assumption that all context words are independent, allowing us to just multiply their probabilities:

$$P(+|w,c_{1:L}) = \prod_{i=1}^{L} \sigma(\mathbf{c_i} \cdot \mathbf{w}) \tag{5.17}$$

$$\log P(+|w,c_{1:L}) = \sum_{i=1}^{L} \log \sigma(\mathbf{c_i} \cdot \mathbf{w}) \tag{5.18}$$

In summary, skip-gram trains a probabilistic classifier that, given a test target word w and its context window of L words $c_{1:L}$, assigns a probability based on how similar this context window is to the target word. The probability is based on applying the

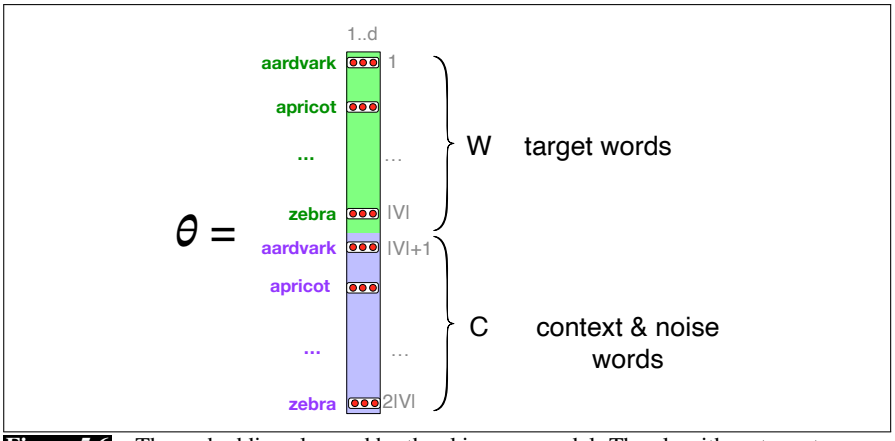


Figure 5.6 The embeddings learned by the skipgram model. The algorithm stores two embeddings for each word, the target embedding (sometimes called the input embedding) and the context embedding (sometimes called the output embedding). The parameter θ that the algorithm learns is thus a matrix of $2|V|$ vectors, each of dimension d, formed by concatenating two matrices, the target embeddings **W** and the context+noise embeddings **C**.

logistic (sigmoid) function to the dot product of the embeddings of the target word with each context word. To compute this probability, we just need embeddings for each target word and context word in the vocabulary.

Fig. 5.6 shows the intuition of the parameters we'll need. Skip-gram actually stores two embeddings for each word, one for the word as a target, and one for the word considered as context. Thus the parameters we need to learn are two matrices **W** and **C**, each containing an embedding for every one of the $|V|$ words in the vocabulary V.[2] Let's now turn to learning these embeddings (which is the real goal of training this classifier in the first place).

5.5.2 Learning skip-gram embeddings

The learning algorithm for skip-gram embeddings takes as input a corpus of text, and a chosen vocabulary size N. It begins by assigning a random embedding vector for each of the N vocabulary words, and then proceeds to iteratively shift the embedding of each word w to be more like the embeddings of words that occur nearby in texts, and less like the embeddings of words that don't occur nearby. Let's start by considering a single piece of training data:

```
... lemon,  a [tablespoon of apricot jam,      a] pinch ...
               c1         c2    w     c3       c4
```

This example has a target word w (apricot), and 4 context words in the $L = \pm 2$ window, resulting in 4 positive training instances (on the left below):

positive examples +

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

w	c_{neg}	w	c_{neg}
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

[2] In principle the target matrix and the context matrix could use different vocabularies, but we'll simplify by assuming one shared vocabulary V.

For training a binary classifier we also need negative examples. In fact skip-gram with negative sampling (SGNS) uses more negative examples than positive examples (with the ratio between them set by a parameter k). So for each of these (w, c_{pos}) training instances we'll create k negative samples, each consisting of the target w plus a 'noise word' c_{neg}. A noise word is a random word from the lexicon, constrained not to be the target word w. The table right above shows the setting where $k = 2$, so we'll have 2 negative examples in the negative training set $-$ for each positive example w, c_{pos}.

The noise words are chosen according to their weighted unigram probability $p_\alpha(w)$, where α is a weight. If we were sampling according to unweighted probability $P(w)$, it would mean that with unigram probability $P(\text{"}the\text{"})$ we would choose the word *the* as a noise word, with unigram probability $P(\text{"}aardvark\text{"})$ we would choose *aardvark*, and so on. But in practice it is common to set $\alpha = 0.75$, i.e. use the weighting $P_{\frac{3}{4}}(w)$:

$$P_\alpha(w) = \frac{count(w)^\alpha}{\sum_{w'} count(w')^\alpha} \tag{5.19}$$

Setting $\alpha = .75$ gives better performance because it gives rare noise words slightly higher probability: for rare words, $P_\alpha(w) > P(w)$. To illustrate this intuition, it might help to work out the probabilities for an example with $\alpha = .75$ and two events, $P(a) = 0.99$ and $P(b) = 0.01$:

$$\begin{aligned} P_\alpha(a) &= \frac{.99^{.75}}{.99^{.75} + .01^{.75}} = 0.97 \\ P_\alpha(b) &= \frac{.01^{.75}}{.99^{.75} + .01^{.75}} = 0.03 \end{aligned} \tag{5.20}$$

Thus using $\alpha = .75$ increases the probability of the rare event b from 0.01 to 0.03.

Given the set of positive and negative training instances, and an initial set of embeddings, the goal of the learning algorithm is to adjust those embeddings to

- Maximize the similarity of the target word, context word pairs (w, c_{pos}) drawn from the positive examples
- Minimize the similarity of the (w, c_{neg}) pairs from the negative examples.

If we consider one word/context pair (w, c_{pos}) with its k noise words $c_{neg_1} \ldots c_{neg_k}$, we can express these two goals as the following loss function L to be minimized (hence the $-$); here the first term expresses that we want the classifier to assign the real context word c_{pos} a high probability of being a neighbor, and the second term expresses that we want to assign each of the noise words c_{neg_i} a high probability of being a non-neighbor, all multiplied because we assume independence:

$$\begin{aligned} L(w, c_{pos}, c_{neg*}) &= -\log \left[P(+|w, c_{pos}) \prod_{i=1}^{k} P(-|w, c_{neg_i}) \right] \\ &= -\left[\log P(+|w, c_{pos}) + \sum_{i=1}^{k} \log P(-|w, c_{neg_i}) \right] \\ &= -\left[\log P(+|w, c_{pos}) + \sum_{i=1}^{k} \log \left(1 - P(+|w, c_{neg_i})\right) \right] \\ &= -\left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^{k} \log \sigma(-c_{neg_i} \cdot w) \right] \end{aligned} \tag{5.21}$$

That is, we want to maximize the dot product of the word with the actual context words, and minimize the dot products of the word with the k negative sampled non-neighbor words.

We minimize this loss function using stochastic gradient descent. Fig. 5.7 shows the intuition of one step of learning.

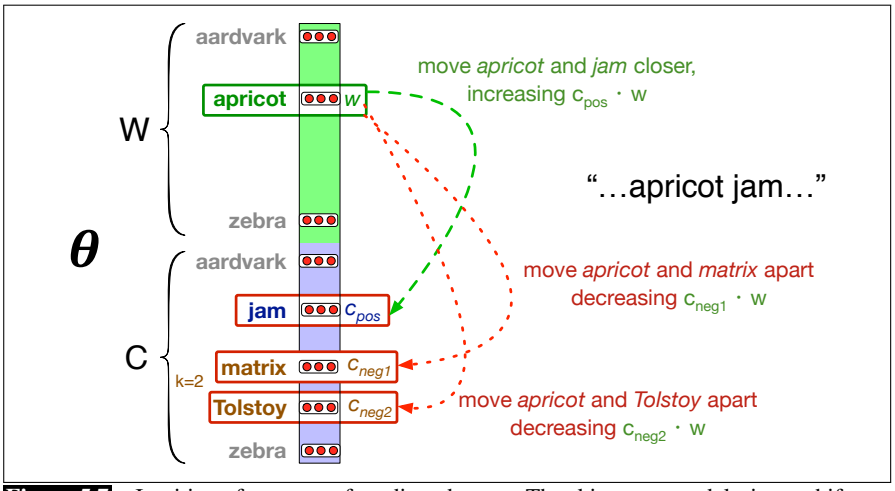


Figure 5.7 Intuition of one step of gradient descent. The skip-gram model tries to shift embeddings so the target embeddings (here for *apricot*) are closer to (have a higher dot product with) context embeddings for nearby words (here *jam*) and further from (lower dot product with) context embeddings for noise words that don't occur nearby (here *Tolstoy* and *matrix*).

To get the gradient, we need to take the derivative of Eq. 5.21 with respect to the different embeddings. It turns out the derivatives are the following (we leave the proof as an exercise at the end of the chapter):

$$\frac{\partial L}{\partial c_{pos}} = [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1]\mathbf{w} \tag{5.22}$$

$$\frac{\partial L}{\partial c_{neg_i}} = [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w})]\mathbf{w} \tag{5.23}$$

$$\frac{\partial L}{\partial w} = [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1]\mathbf{c}_{pos} + \sum_{i=1}^{k}[\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w})]\mathbf{c}_{neg_i} \tag{5.24}$$

The update equations going from time step t to $t+1$ in stochastic gradient descent are thus:

$$\mathbf{c}_{pos}^{t+1} = \mathbf{c}_{pos}^{t} - \eta[\sigma(\mathbf{c}_{pos}^{t} \cdot \mathbf{w}^{t}) - 1]\mathbf{w}^{t} \tag{5.25}$$

$$\mathbf{c}_{neg_i}^{t+1} = \mathbf{c}_{neg_i}^{t} - \eta[\sigma(\mathbf{c}_{neg_i}^{t} \cdot \mathbf{w}^{t})]\mathbf{w}^{t} \tag{5.26}$$

$$\mathbf{w}^{t+1} = \mathbf{w}^{t} - \eta\left[[\sigma(\mathbf{c}_{pos}^{t} \cdot \mathbf{w}^{t}) - 1]\mathbf{c}_{pos}^{t} + \sum_{i=1}^{k}[\sigma(\mathbf{c}_{neg_i}^{t} \cdot \mathbf{w}^{t})]\mathbf{c}_{neg_i}^{t}\right] \tag{5.27}$$

Just as in logistic regression, then, the learning algorithm starts with randomly initialized $\mathbf{W}$ and $\mathbf{C}$ matrices, and then walks through the training corpus using gradient descent to move $\mathbf{W}$ and $\mathbf{C}$ so as to minimize the loss in Eq. 5.21 by making the updates in (Eq. 5.25)-(Eq. 5.27).

target embedding
context embedding

Recall that the skip-gram model learns **two** separate embeddings for each word i: the **target embedding** $\mathbf{w}_i$ and the **context embedding** $\mathbf{c}_i$, stored in two matrices, the **target matrix** $\mathbf{W}$ and the **context matrix** $\mathbf{C}$. It's common to just add them together, representing word i with the vector $\mathbf{w}_i + \mathbf{c}_i$. Alternatively we can throw away the $\mathbf{C}$ matrix and just represent each word i by the vector $\mathbf{w}_i$.

As with the simple count-based methods like tf-idf, the context window size affects the performance of skip-gram embeddings, and experiments often tune the context window size parameter on a devset.

5.5.3 Other kinds of static embeddings

fasttext

There are many kinds of static embeddings. An extension of word2vec, **fasttext** (Bojanowski et al., 2017), addresses a problem with word2vec as we have presented it so far: it has no good way to deal with **unknown words**—words that appear in a test corpus but were unseen in the training corpus. A related problem is word sparsity, such as in languages with rich morphology, where some of the many forms for each noun and verb may only occur rarely. Fasttext deals with these problems by using subword models, representing each word as itself plus a bag of constituent n-grams, with special boundary symbols < and > added to each word. For example, with $n = 3$ the word *where* would be represented by the sequence <where> plus the character n-grams:

```
<wh, whe, her, ere, re>
```

Then a skipgram embedding is learned for each constituent n-gram, and the word *where* is represented by the sum of all of the embeddings of its constituent n-grams. Unknown words can then be represented only by the sum of the constituent n-grams. A fasttext open-source library, including pretrained embeddings for 157 languages, is available at https://fasttext.cc.

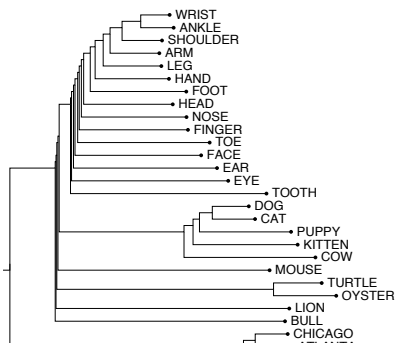
Another very widely used static embedding model is GloVe (Pennington et al., 2014), short for Global Vectors, because the model is based on capturing global corpus statistics. GloVe is based on ratios of probabilities from the word-word co-occurrence matrix.

It turns out that dense embeddings like word2vec actually have an elegant mathematical relationship with count-based embeddings, in which word2vec can be seen as implicitly optimizing a function of a count matrix with a particular (PPMI) weighting (Levy and Goldberg, 2014c).

5.6 Visualizing Embeddings

> "I see well in many dimensions as long as the dimensions are around two."
>
> The late economist Martin Shubik

Visualizing embeddings is an important goal in helping understand, apply, and improve these models of word meaning. But how can we visualize a (for example) 100-dimensional vector?



The simplest way to visualize the meaning of a word w embedded in a space is to list the most similar words to w by sorting the vectors for all words in the vocabulary by their cosine with the vector for w. For example the 7 closest words to *frog* using a particular set of embeddings computed

with the GloVe algorithm are: *frogs*, *toad*, *litoria*, *leptodactylidae*, *rana*, *lizard*, and *eleutherodactylus* (Pennington et al., 2014).

Yet another visualization method is to use a clustering algorithm to show a hierarchical representation of which words are similar to others in the embedding space. The figure on the left uses hierarchical clustering of some embedding vectors for nouns as a visualization method (Rohde et al., 2006).

Probably the most common visualization method, however, is to project the 100 dimensions of a word down into 2 dimensions. Fig. 5.1 showed one such visualization, as does Fig. 5.10, using a projection method called t-SNE (van der Maaten and Hinton, 2008).

5.7 Semantic properties of embeddings

In this section we briefly summarize some of the semantic properties of embeddings that have been studied.

Different types of similarity or association: One parameter of vector semantic models is the size of the context window used to collect counts. This is generally between 1 and 10 words on each side of the target word (for a total context of 2-20 words).

The choice depends on the goals of the representation. Shorter context windows tend to lead to representations that are a bit more syntactic, since the information is coming from immediately nearby words. When the vectors are computed from short context windows, the most similar words to a target word w tend to be semantically similar words with the same parts of speech. When vectors are computed from long context windows, the highest cosine words to a target word w tend to be words that are topically related but not similar.

For example Levy and Goldberg (2014a) showed that using skip-gram with a window of ± 2, the most similar words to the word *Hogwarts* (from the *Harry Potter* series) were names of other fictional schools: *Sunnydale* (from *Buffy the Vampire Slayer*) or *Evernight* (from a vampire series). With a window of ± 5, the most similar words to *Hogwarts* were other words topically related to the *Harry Potter* series: *Dumbledore*, *Malfoy*, and *half-blood*.

first-order co-occurrence

second-order co-occurrence

It's also often useful to distinguish two kinds of similarity or association between words (Schütze and Pedersen, 1993). Two words have **first-order co-occurrence** (sometimes called **syntagmatic association**) if they are typically nearby each other. Thus *wrote* is a first-order associate of *book* or *poem*. Two words have **second-order co-occurrence** (sometimes called **paradigmatic association**) if they have similar neighbors. Thus *wrote* is a second-order associate of words like *said* or *remarked*.

parallelogram model

Analogy/Relational Similarity: Another semantic property of embeddings is their ability to capture relational meanings. In an important early vector space model of cognition, Rumelhart and Abrahamson (1973) proposed the **parallelogram model** for solving simple analogy problems of the form *a is to b as a* is to what?*. In such problems, a system is given a problem like *apple:tree::grape:?*, i.e., *apple is to tree as grape is to* _____, and must fill in the word *vine*. In the parallelogram model, il-

lustrated in Fig. 5.8, the vector from the word *apple* to the word *tree* ($= \overrightarrow{\text{tree}} - \overrightarrow{\text{apple}}$) is added to the vector for *grape* ($\overrightarrow{\text{grape}}$); the nearest word to that point is returned.

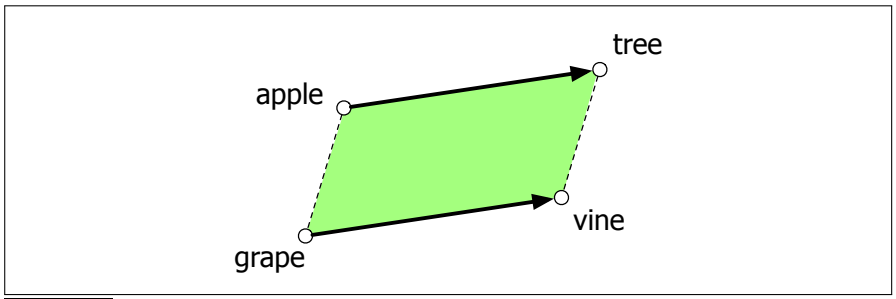


Figure 5.8 The parallelogram model for analogy problems (Rumelhart and Abrahamson, 1973): the location of $\overrightarrow{\text{vine}}$ can be found by subtracting $\overrightarrow{\text{apple}}$ from $\overrightarrow{\text{tree}}$ and adding $\overrightarrow{\text{grape}}$.

In early work with sparse embeddings, scholars showed that sparse vector models of meaning could solve such analogy problems (Turney and Littman, 2005), but the parallelogram method received more modern attention because of its success with word2vec or GloVe vectors (Mikolov et al. 2013c, Levy and Goldberg 2014b, Pennington et al. 2014). For example, the result of the expression $\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}}$ is a vector close to $\overrightarrow{\text{queen}}$. Similarly, $\overrightarrow{\text{Paris}} - \overrightarrow{\text{France}} + \overrightarrow{\text{Italy}}$ results in a vector that is close to $\overrightarrow{\text{Rome}}$. The embedding model thus seems to be extracting representations of relations like MALE-FEMALE, or CAPITAL-CITY-OF, or even COMPARATIVE/SUPERLATIVE, as shown in Fig. 5.9 from GloVe.

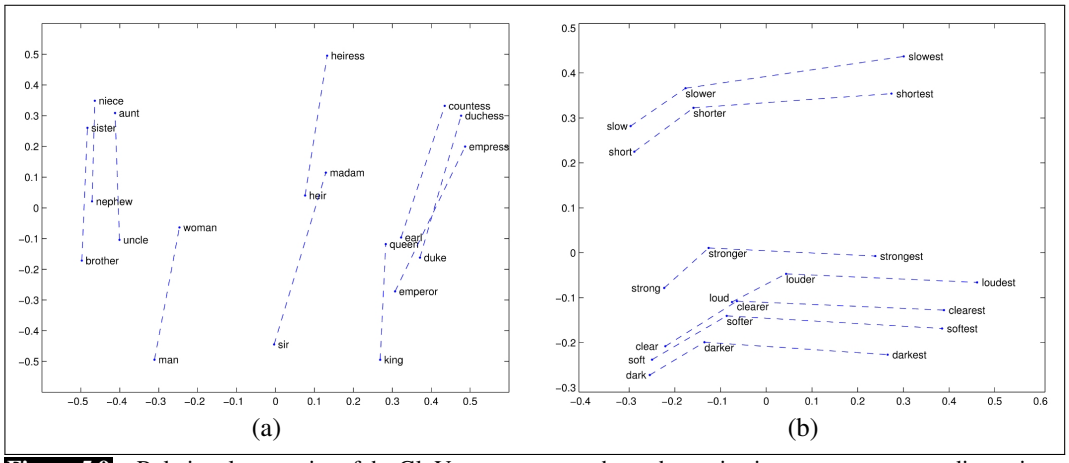


Figure 5.9 Relational properties of the GloVe vector space, shown by projecting vectors onto two dimensions. (a) $\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}}$ is close to $\overrightarrow{\text{queen}}$. (b) offsets seem to capture comparative and superlative morphology (Pennington et al., 2014).

For an $\mathbf{a} : \mathbf{b} :: \mathbf{a}^* : \mathbf{b}^*$ problem, meaning the algorithm is given vectors $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{a}^*$ and must find $\mathbf{b}^*$, the parallelogram method is thus:

$$\hat{\mathbf{b}}^* = \underset{\mathbf{x}}{\text{argmin}}\ \text{distance}(\mathbf{x}, \mathbf{b} - \mathbf{a} + \mathbf{a}^*) \tag{5.28}$$

with some distance function, such as Euclidean distance.

There are some caveats. For example, the closest value returned by the parallelogram algorithm in word2vec or GloVe embedding spaces is usually not in fact

b* but one of the 3 input words or their morphological variants (i.e., *cherry:red :: potato:x* returns *potato* or *potatoes* instead of *brown*), so these must be explicitly excluded. Furthermore while embedding spaces perform well if the task involves frequent words, small distances, and certain relations (like relating countries with their capitals or verbs/nouns with their inflected forms), the parallelogram method with embeddings doesn't work as well for other relations (Linzen 2016, Gladkova et al. 2016, Schluter 2018, Ethayarajh et al. 2019a), and indeed Peterson et al. (2020) argue that the parallelogram method is in general too simple to model the human cognitive process of forming analogies of this kind.

5.7.1 Embeddings and Historical Semantics

Embeddings can also be a useful tool for studying how meaning changes over time, by computing multiple embedding spaces, each from texts written in a particular time period. For example Fig. 5.10 shows a visualization of changes in meaning in English words over the last two centuries, computed by building separate embedding spaces for each decade from historical corpora like Google n-grams (Lin et al., 2012b) and the Corpus of Historical American English (Davies, 2012).

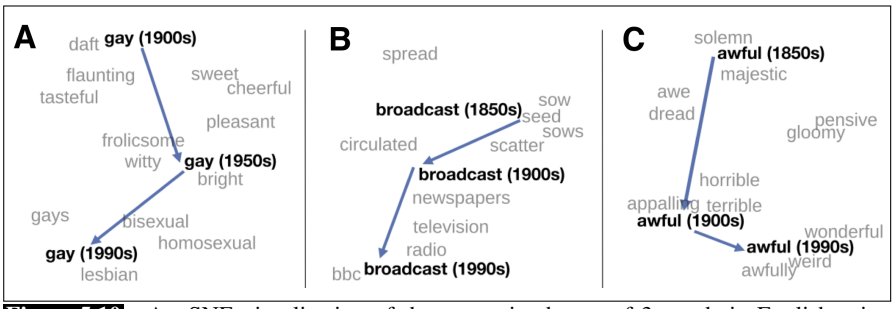


Figure 5.10 A t-SNE visualization of the semantic change of 3 words in English using word2vec vectors. The modern sense of each word, and the grey context words, are computed from the most recent (modern) time-point embedding space. Earlier points are computed from earlier historical embedding spaces. The visualizations show the changes in the word *gay* from meanings related to "cheerful" or "frolicsome" to referring to homosexuality, the development of the modern "transmission" sense of *broadcast* from its original sense of sowing seeds, and the pejoration of the word *awful* as it shifted from meaning "full of awe" to meaning "terrible or appalling" (Hamilton et al., 2016b).

5.8 Bias and Embeddings

In addition to their ability to learn word meaning from text, embeddings, alas, also reproduce the implicit biases and stereotypes that were latent in the text. As the prior section just showed, embeddings can roughly model relational similarity: 'queen' as the closest word to 'king' - 'man' + 'woman' implies the analogy *man:woman::king:queen*. But these same embedding analogies also exhibit gender stereotypes. For example Bolukbasi et al. (2016) find that the closest occupation to 'computer programmer' - 'man' + 'woman' in word2vec embeddings trained on news text is 'homemaker', and that the embeddings similarly suggest the analogy 'father' is to 'doctor' as 'mother' is to 'nurse'. This could result in what Crawford (2017) and Blodgett et al. (2020) call an **allocational harm**, when a system allo-

allocational harm